Locust Installation and Configuration Guide

A comprehensive guide for setting up Locust, writing modular test scenarios in Python, and evaluating web service performance metrics.

1. Prerequisites & Environment Setup

Locust requires Python 3.8 or later. It is strongly recommended to isolate dependencies inside an independent virtual environment to prevent package conflicts.

Step 1: Verify Python Installation

Ensure Python 3 and its package manager are present on your system:

```
None
python3 --version
python3 -m pip --version
```

Step 2: Initialize Virtual Environment

Navigate to your project directory and generate an isolated environment named `.venv`:

```
None
mkdir locust-load-testing
cd locust-load-testing
python3 -m venv .venv
```

Step 3: Activate the Environment

Activate the environment before installing packages or executing test scripts:

- **macOS / Linux:**

```
None
source .venv/bin/activate
```

- **Windows (Command Prompt):**

```
None
.venv\Scripts\activate.bat
```

- **Windows (PowerShell):**

```
None
.venv\Scripts\Activate.ps1
```

Once activated, your terminal prompt will display the `(.venv)` prefix.

2. Installing Locust

Install Locust inside the activated virtual environment using standard `pip` workflows.

Installation Command

```
None
pip install --upgrade pip
pip install locust
```

Verify Installation

Confirm that the executable is accessible and verify the installed release version:h
python3 -m locust --version

```
None

A standard output format will resemble: `locust 2.x.x from ... (python 3.x.x)`.

## 3. Creating the Locustfile
```

Locust tests are defined purely in Python code. Test scenarios describe user behaviors through classes extending `HttpUser`.

Create a file named `locustfile.py` in your working directory.

Core Components
* **`HttpUser`**: Provides an HTTP client (`self.client`) configured to retain cookies, manage connections, and execute typical requests (`GET`, `POST`, etc.).
* **`@task` Decorator**: Denotes methods as user activities. An optional numeric weight specifies task execution frequency (e.g., `@task(3)` runs three times more frequently than `@task(1)`).
* **`wait_time`**: Defines pausing behavior between task iterations (e.g., `between(min, max)` simulates realistic user reading or processing delays).

Sample Implementation (`locustfile.py`)

```python
import logging
from locust import HttpUser, task, between

# Configure logger for structured trace output
logger = logging.getLogger("locust_test")
logger.setLevel(logging.INFO)

class WebsiteUser(HttpUser):
    # Simulate a delay between 1 and 3 seconds after each task
    wait_time = between(1.0, 3.0)

    @task(3)
    def load_homepage(self):
        """Simulates loading the main index page."""
        response = self.client.get("/")
        logger.info(f"Loaded index: Status {response.status_code}")

    @task(1)
    def load_status_endpoint(self):
        """Simulates querying a lightweight status/health endpoint."""
        with self.client.get("/status", catch_response=True) as response:
            if response.status_code == 200:
                response.success()
```

```
                logger.info("Status check verified successfully.")
            else:
                response.failure(f"Unexpected status code:
{response.status_code}")
                logger.warning(f"Status check failed: {response.status_code}")
```

4. Running Locust

Launch the test runner using either standard entry point.

Standard Invocation

If your script is named `locustfile.py`, Locust automatically detects it:

```
None
locust
```

Explicit File Specification

To target an arbitrary file or module path:

```
None
locust -f locustfile.py
```

Module Execution

Alternatively, run the module directly through the Python binary:

```
None
python3 -m locust -f locustfile.py
```

Once running, the command line interface will output a web server notification:
`[INFO/MainProcess] Starting web interface at http://0.0.0.0:8089`

Open your web browser and navigate to `http://localhost:8089` to interact with the dashboard.

5. Configuring Web UI Parameters

Upon opening http://localhost:8089, a configuration form will prompt you for the test parameters before beginning execution:

Parameter	Description	Recommended Starting Value
Number of Users	Total concurrent user instances to generate.	`10` to `50` (gradual scaling)
Spawn Rate	Number of users added per second until reaching the peak.	`1` to `5` users/sec
Host URL	Base target domain/IP against which relative paths are resolved.	`https://example.com` or local service

Click **Start Swarming** to begin generating load against the target application.

6. Basic Load Metrics Analysis

The Locust Web UI offers real-time visualization and statistical tables summarizing system performance:

Metric	Definition	Engineering Focus
Requests	The cumulative total count of HTTP requests issued to an endpoint.	Verifies test load distribution aligns with `@task` weighting.

Metric	Definition	Engineering Focus
Response Time	Duration required to transmit the request and receive the full response (ms).	Monitor median, 95th percentile, and 99th percentile for latency spikes.
RPS (Requests Per Second)	Current real-time throughput delivered by the running swarm.	Indicates whether application throughput plateaus or bottlenecks under higher load.
Failures	Count of requests that timed out, encountered HTTP 4xx/5xx codes, or failed asserts.	Should remain at zero under standard nominal operating thresholds.